

用于并行函数调用的 LLM 编译器

Sehoon Kim^{*1} Suhong Moon^{*1} Ryan Tabrizi¹ Nicholas Lee¹ Michael W. Mahoney^{1,2,3}
Kurt Keutzer¹ Amir Gholami^{1,2}

摘要

近期 LLM 的推理能力使它们能够调用外部函数，弥补知识截止、算术能力不足或无法访问私有数据等内在限制；进而根据上下文选择并协调多个函数，处理更复杂的问题。然而，现有函数调用方法通常逐个函数顺序进行推理与行动，造成较高延迟和成本，有时也导致不准确行为。为此，我们提出 LLMCompiler，通过并行执行函数，高效编排多个函数调用。受经典编译器原理启发，LLMCompiler 以三个组件支持并行函数调用：(i) 函数调用规划器，制定执行计划；(ii) 任务获取单元，分发调用任务；(iii) 执行器，并行执行任务。LLMCompiler 自动生成经过优化的函数调用编排，既适用于开源模型，也适用于闭源模型。我们在具有不同调用模式的一系列任务上评估该系统。相比 ReAct，持续获得最高 3.7× 的延迟加速、最高 6.7× 的成本降低，以及最高 ~9% 的准确率提升。代码见 <https://github.com/SqueezeAILab/LLMCompiler>。

1. 引言

大语言模型 (LLM) 推理能力的发展，使应用范围从内容生成拓展到复杂问题求解 (Besta et al., 2023; Chen et al., 2023b; Gao et al., 2022; Kojima et al., 2023; Wang et al., 2023b; Wei et al., 2022; Yang et al., 2022; Yao et al., 2023b; Zhou et al., 2023b)。近期研究也表明，这种能力能够提高复杂、逻辑任务的求解准确率。它还支持函数，即工具调用：LLM 可以调用提供的函数，并利用输出完成任务。这些函数可以是执行算术运算的简单计算器，也可以是更复杂、基于 LLM 的函数。

将各种工具和函数调用整合进 LLM，可能根本改变基于 LLM 的软件开发方式。不过，这带来一个重要挑战：怎样才能最有效地组合多个函数调用？ReAct (Yao et al., 2022) 提出一种重要方法：LLM 调用函数，分析结果，再推理下一步行动，触发后续调用。以图 1

左侧的简单问题为例，用户询问 Scott Derrickson 与 Ed Wood 是否具有相同国籍。ReAct 先分析查询，决定使用搜索工具查找 Scott Derrickson；随后将搜索结果，即观察，拼回原提示，让 LLM 推理下一步行动，再调用搜索工具收集 Ed Wood 的信息。

ReAct 是支持函数调用的先驱工作，已经集成到多个框架 (Langchain; Liu, 2022)。但要扩展到复杂应用，仍需大量优化，因为它顺序执行函数调用，逐个观察结果进行推理。ReAct 及其扩展智能体系统 (Khot et al., 2023; Qin et al., 2023; Ruan et al., 2023b; Sumers et al., 2023; Yao et al., 2023b) 会为每次推理与行动重复调用 LLM，加上顺序函数执行，造成延迟和成本低效。虽然基于观察的动态推理在某些情况有益，但拼接中间调用输出可能干扰 LLM 执行流程，降低准确率 (Xu et al., 2023)。常见失败包括重复调用同一函数，原 ReAct 论文 (Yao et al., 2022) 也指出这一问题，以及根据部分中间结果过早停止；第 5.1 节和附录 A 将进一步讨论。

为解决这一问题，我们借鉴经典编译器。传统程序语言中的指令执行优化已得到广泛研究，其中一项关键技术是识别可并行执行的指令，并管理其依赖。类似地，可以为 LLM 函数调用设计编译器，高效编排不同调用及其依赖。这与近期将 LLM 与计算机系统对应起来的研究思路相近 (Karpathy, 2023; Packer et al., 2023)。因此，我们提出 LLMCompiler，一个跨模型、跨负载支持并行多工具执行的框架。据我们所知，它是首个优化 LLM 函数调用编排、同时改善延迟、成本和准确率的框架；准确率收益来自减少中间函数输出的干扰。具体贡献如下：

- 提出 LLMCompiler，优化 LLM 并行函数调用性能。主要通过三个组件实现：(i) 函数调用规划器，识别执行流程 (第 3.1 节)；(ii) 任务获取单元，并行分发调用 (第 3.2 节)；(iii) 执行器，使用相应函数执行所分发任务 (第 3.3 节)。
- 使用 HotpotQA (Yang et al., 2018) 和电影推荐基准 (Srivastava et al., 2022)，评估易于完全并行的调用模式；相比 ReAct，分别获得 1.80×/3.74× 加速，以及 3.37×/6.73× 成本降低 (第 5.1 节)。
- 为测试复杂模式，提出新基准 ParallelQA，包含多种非平凡调用模式。相比 ReAct，最高获得 2.27× 加速、4.65× 成本降低和 9% 准确率提升 (第 5.2 节)。

^{*} 同等贡献 ¹UC Berkeley ²ICSI ³LBNL. 通讯作者: Amir Gholami <amirgh@berkeley.edu>.

第 41 届 International Conference on Machine Learning 会议论文集, Vienna, Austria. PMLR 235, 2024. 2024 年版权归作者所有。

Question: Were Scott Derrickson and Ed Wood of the same nationality?

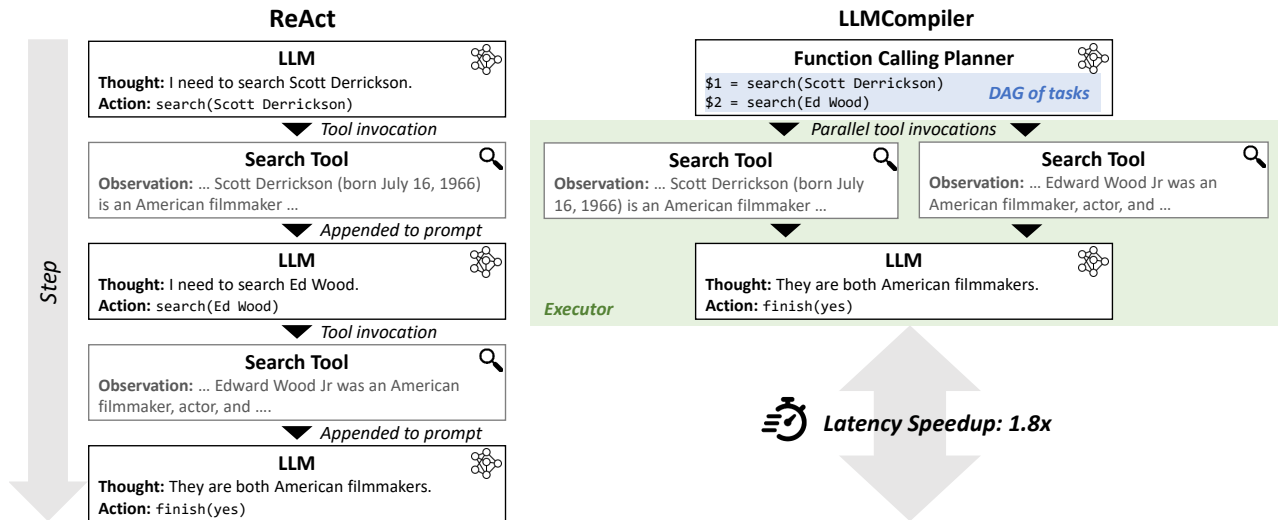


图 1. 对比 LLMCompiler 与 ReAct (Yao et al., 2022) 的运行行为, 采用 HotpotQA 基准 (Yang et al., 2018) 的示例问题。在 LLMCompiler (右) 中, 规划器先将查询分解为多个具有依赖关系的任务; 执行器遵守依赖, 并行执行多个任务; 最后汇总工具执行得到的全部观察结果, 生成最终回答。ReAct 等现有框架 (左) 顺序执行工具, 延迟更长。在这个 HotpotQA 示例中, LLMCompiler 获得 1.8 $\times$ 的延迟加速。为便于示意, 这里采用可进行两路并行的问题; LLMCompiler 也能管理更复杂的依赖模式 (图 2 与第 5 节)。图内文字译释: Question 为问题: “Scott Derrickson 与 Ed Wood 是否具有相同国籍?”; Step 为步骤; Thought 为思考, 三段分别为 “我需要搜索 Scott Derrickson” “我需要搜索 Ed Wood” “他们都是美国电影制作人”; Action 为动作; Tool invocation 为工具调用; Appended to prompt 为追加到提示; Search Tool 为搜索工具; Observation 为观察, 两个结果分别说明 Scott Derrickson (生于 1966 年 7 月 16 日) 是美国电影制作人、Edward Wood Jr 是美国电影制作人和演员; Function Calling Planner 为函数调用规划器; DAG of tasks 为任务 DAG; Parallel tool invocations 为并行工具调用; Executor 为执行器; Latency Speedup: 1.8 $\times$ 为时延加速比 1.8 倍。

- 评估动态重新规划能力: 通过执行器到规划器的反馈循环实现。24 点游戏 (Yao et al., 2023b) 需要根据中间结果反复重新规划; 在该任务中, 相比思维树 (Tree-of-Thoughts), LLMCompiler 获得 2 $\times$ 加速 (第 5.3 节)。
- 展示在交互决策环境中的有效、高效探索。在 WebShop 上, 相比基线, LLMCompiler 最高获得 101.7 $\times$ 加速和 25.7% 的成功率提升 (第 5.4 节)。

2. 相关工作

2.1. LLM 的时延优化

多项研究从模型设计 (Chen et al., 2023a; Dettmers et al., 2023; Frantar & Alistarh, 2023; Frantar et al., 2022; Kim et al., 2023; 2024; Kwon et al., 2022; Leviathan et al., 2023; Lin et al., 2023) 和系统 (tgi; trt; Kwon et al., 2023; Yu et al., 2022) 两方面优化 LLM 推理效率。不过, 应用层的优化还较少受到研究。在使用黑盒 LLM 模型及服务、难以修改模型和底层推理流水线的实际场景中, 应用层优化尤其关键。

Skeleton-of-Thought (Ning et al., 2023) 近期提出通过应用层的并行解码降低时延。它分为两个步骤: 先生成回答骨架, 再并行执行各个骨架条目。然而, 该方法主要面向易于完全并行的工作负载, 假定骨架任务彼此没有依赖, 因此不能支持任务之间存在内在依赖的问题。这一限制妨碍了它在编程 (Austin et al., 2021; Chen

et al., 2021; Hendrycks et al., 2021a; Madaan et al., 2023) 或数学 (Hendrycks et al., 2021b;c) 等复杂场景中的应用, 该论文 (Ning et al., 2023) 也明确指出了这一点。LLMCompiler 将输入查询转换为一组带有相互依赖关系的任务, 从而扩大了可处理的问题范围。

与本工作同期, OpenAI 在其 1106 版本中引入了并行函数调用功能, 通过同时生成多个函数调用来改进用户查询的处理 (OpenAI, 2023)。虽然该功能有望缩短 LLM 的执行时间, 但它仅适用于 OpenAI 的专有模型, 因而存在限制。随着开源 LLM 不断增多, 以及用于微调和定制的参数高效训练技术 (Houlsby et al., 2019; Hu et al., 2022; Lester et al., 2021) 的发展, 使用开源模型的需求日益增长。LLMCompiler 为开源模型提供高效的并行函数调用; 此外, 如第 5 节所示, 它还有可能获得更低的时延与成本。

2.2. 先规划再求解的策略

多项研究 (Hao et al., 2023; Patel et al., 2022; Press et al., 2023; Wolfson et al., 2020; Zhou et al., 2023b) 探索了提示方法, 将复杂查询分解到不同粒度后逐一求解, 以提高 LLM 在推理任务中的表现。具体而言, Decomposed Prompting (Khot et al., 2023) 将复杂任务分成更简单的子任务, 并用各自专门的提示指导 LLM 优化求解。Step-Back Prompting (Zheng et al., 2023) 让 LLM 从细节中抽象出高层概念, 以增强其在多种

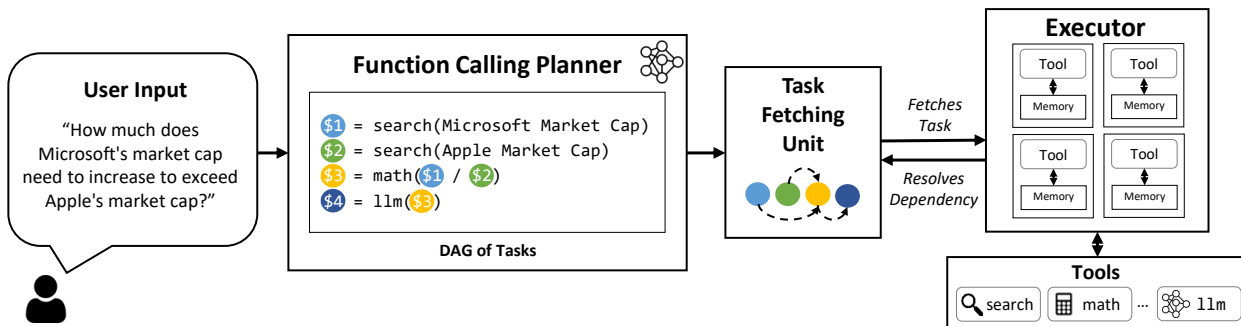


图 2. LLMCompiler 框架概览。函数调用规划器生成包含任务及其依赖关系的 DAG。任务获取单元根据依赖关系，将这些任务分派给执行器并行执行。在本例中，任务 \$1 和 \$2 一同被获取，以并行执行两个相互独立的搜索任务。每个任务执行完毕后，将结果回传给任务获取单元；任务获取单元把依赖任务中的占位变量（例如任务 \$3 中的变量 \$1 和 \$2）替换为实际值，从而解除这些任务的阻塞。全部任务执行完毕后，向用户返回最终答案。图内文字译释：User Input 为用户输入，问题为“Microsoft 的市值需要增长多少才能超过 Apple 的市值？”；Function Calling Planner 为函数调用规划器；DAG of Tasks 为任务 DAG；Task Fetching Unit 为任务获取单元；Fetches Task 为获取任务；Resolves Dependency 为解析依赖；Executor 为执行器；Tools/Tool 为工具；Memory 为内存；search 的两个查询分别是 Microsoft 与 Apple 的市值，math 为数学计算，llm 为 LLM 调用。

任务上的推理能力。Plan-and-Solve Prompting (Wang et al., 2023a) 将多步推理任务拆为子任务，无须人工设计逐步提示即可减少错误并提高准确率。不过，这些方法主要关注推理基准的准确率。LLMCompiler 则借助规划器识别查询中可并行的模式，目标是在保持准确率的同时降低时延。

除上述工作外，TPTU (Ruan et al., 2023a)、HuggingGPT (Shen et al., 2023) 和 ViperGPT (Surís et al., 2023) 也提出了端到端的先规划再求解框架。LLMCompiler 提供适用于更广泛问题的通用框架，实现高效、准确的函数调用。它具备三方面能力：(i) 规划与重新规划；(ii) 并行执行；(iii) 处理更多问题领域。附录 F 将进一步讨论这些能力。

另一项值得关注的工作是 ReWOO (Xu et al., 2023)。它利用规划器，把推理过程与执行、观察阶段分离，以减少相对于 ReAct 的 Token 用量和成本。本方法与 ReWOO 在多方面有所不同。首先，LLMCompiler 支持并行函数调用，可以同时降低时延和成本。其次，LLMCompiler 支持动态重新规划；对于无法在开始时静态确定执行流程的问题，这一能力十分重要（第 5.3 节）。

2.3. 工具增强的 LLM

LLM 推理能力的增强，使其能够调用用户提供的函数，并利用函数输出来有效完成任务。附录 C.1 对这一主题展开了详细讨论。

3. 方法

我们用图 2 中一个具有两路并行性的简单示例说明 LLMCompiler 的组成。为了回答“Microsoft 的市值需要增长多少才能超过 Apple 的市值？”，LLM 首先需要分别在网上搜索两家公司的市值，再执行除法运算。现

有框架（包括 ReAct）会顺序执行这些任务，但两个搜索任务显然可以并行。关键在于如何自动判断哪些任务能够并行、哪些任务存在依赖，从而相应地编排不同任务的执行。LLMCompiler 的系统通过以下三个组件完成这一过程：函数调用规划器（第 3.1 节）生成任务序列及其依赖关系；任务获取单元（第 3.2 节）根据中间结果替换参数并获取任务；执行器（第 3.3 节）使用对应工具执行任务。使用 LLMCompiler 时，用户只需提供工具定义，也可以为规划器提供上下文示例；第 4.1 节将进一步讨论这一点。

3.1. 函数调用规划器

函数调用规划器负责生成待执行的任务序列及其依赖关系。例如，图 2 中的任务 \$1 和 \$2 是两个独立的搜索，可以并行执行；但任务 \$3 依赖于前两个搜索的结果。因此，规划器利用 LLM 较强的推理能力，自动识别所需任务、输入参数以及任务之间的依赖，实际形成一个表示任务依赖的有向无环图。如果一个任务依赖于前序任务，它的参数中会包含占位变量，例如图 2 中任务 3 的 \$1；这个变量随后将由前序任务的实际输出替换（第 3.2 节）。

LLMCompiler 的规划器利用 LLM 的推理能力，从自然语言输入中分解出任务。为此，规划器 LLM 使用预先定义的提示，指导它构建依赖图并保证语法正确（详见附录 H）。此外，用户需要提供工具定义，也可以提供上下文示例。这些示例详细展示特定问题的任务分解过程，帮助规划器更好地理解规则。第 4.1 节进一步说明 LLMCompiler 所需的用户输入。第 4.2 节还介绍一项规划器优化：每个任务一经生成就以流式方式输出，无须等到整个规划过程完成。

3.2. 任务获取单元

任务获取单元受到现代计算机体系结构中指令获取单元的启发，采用贪心策略：一旦任务满足依赖、已就绪，就立即交给执行器执行（可并行）。它的另一项关键功能，是把规划器最初设置的占位变量替换为前序任务的实际输出。对于图 2 中的示例，任务 \$3 中的变量 \$1 和 \$2 将被替换为 Microsoft 与 Apple 的实际市值。这一过程只需简单的任务获取与排队机制即可实现，无须专门调用 LLM。

3.3. 执行器

执行器异步执行从任务获取单元取得的任务。任务获取单元保证交给执行器的任务彼此独立，因此执行器可以直接并发运行它们。执行器配备用户提供的工具，将每个任务委派给对应工具。这些工具可以是计算器、Wikipedia 搜索或 API 调用等简单函数，也可以是针对某一任务专门设计的 LLM 智能体。如图 2 的执行器模块所示，每个任务都有专用内存来保存中间结果，这与常见顺序框架将观察结果汇总成一个提示的做法相似 (Yao et al., 2022)。任务完成后，其最终结果被作为输入传给依赖它的任务。

3.4. 动态重新规划

在多种应用中，执行图需要根据事先未知的中间结果调整。这类似于程序中的分支：只有在运行时，才会根据满足的分支条件决定执行路径。LLM 函数调用同样可能出现这种动态执行模式。对于简单分支（如 if-else 语句），可以静态编译执行流程，再根据中间结果在运行时选择正确分支。但对于更复杂的分支，依据中间结果重新编译或重新规划可能更合适。

重新规划时，执行器将中间结果发回函数调用规划器；规划器生成新的任务及其依赖关系。这些任务先交给任务获取单元，再交给执行器。如此循环，直到得到所需最终结果并将其返回用户。第 5.3 节给出一个用例：使用 Tree-of-Thoughts 方法求解 24 点游戏。

4. LLMCompiler 的实现细节

4.1. 用户提供的信息

LLMCompiler 需要用户提供以下两类输入：

1. **工具定义**：用户需要指定 LLM 可以使用的工具，包括工具描述和参数规范。这与 ReAct、OpenAI 函数调用等框架的要求基本相同。
2. **规划器的上下文示例**：用户可以选择向 LLMCompiler 提供示例，说明规划器应如何工作。例如，在图 2 的场景中，可以提供示例，展示某些查询预期具有的任务间依赖关系。这些示例帮助规划器 LLM 理解如何使用不同工具，并以正确格式为输入生成适当的依赖图。附录 G 包含本次评测使用的示例。

4.2. 流式规划器

当用户查询涉及许多任务时，规划器可能带来不可忽略的开销：它阻塞任务获取单元和执行器，后两者必须等到规划器输出后才能开始工作。不过，类似现代计算机系统指令流水线的规划器，可以让规划器异步流式输出依赖图，使每个任务在全部依赖得到满足后立即由执行器处理。表 C.1 对比了 LLMCompiler 在不同基准上启用和禁用流式机制时的时延。结果显示，流式机制始终带来时延改善。尤其是在 ParallelQA 基准上，流式机制最多获得 1.3× 的时延收益。这是因为 ParallelQA 使用的 math 工具执行时间较长，可以有效掩盖规划器生成后续任务的时延；HotpotQA 和 Movie Recommendation 使用的 search 工具执行时间较短，因此无法同等程度地掩盖规划时延。

5. 实验结果

本节使用多种模型和问题类型评测 LLMCompiler。我们既使用专有的 GPT 模型，也使用开源的 LLaMA-2 模型；后者展示了 LLMCompiler 为开源模型提供并行函数调用的能力。LLM 能处理多种并行函数调用模式：既包括任务之间没有任何依赖、所有任务可以同时执行的完全并行模式，也包括图 3 所示的更复杂依赖模式。我们还在 24 点游戏基准上评测 LLMCompiler；这一基准需要根据中间结果动态重新规划，展示了框架对动态依赖图的适应能力。最后，我们在 WebShop 基准上应用 LLMCompiler，以展示它在决策任务中的潜力。结果从简单执行模式开始介绍，逐步扩展到更复杂的模式。

5.1. 完全并行的函数调用

最简单的场景是 LLM 针对相互独立的任务反复使用同一工具，例如并行搜索或分析不同主题的信息，正如图 3 中的模式 (a)。虽然这些任务相互独立、可以并行执行，但 ReAct 及现有其他 LLM 方案仍需顺序运行。每次使用工具都频繁调用 LLM，因而增加时延和 Token 用量，图 1 也说明了这一点。本节展示 LLMCompiler 如何识别可并行模式、并发执行独立任务以解决该问题。我们使用以下两个基准：

- **HotpotQA**：评测多跳推理的数据集 (Yang et al., 2018)。我们只使用比较类开发集，其中有 1.5k 个比较两个不同实体的问题，因而呈现两路完全并行的执行模式。图 1 展示了一个问题示例。
- **Movie Recommendation**：该数据集包含 500 个示例，要求从四个候选电影中选出与另外一组四部电影最相似的一部，因此呈现八路完全并行模式 (Srivastava et al., 2022)。

实验设置。我们以 ReAct 为基线，与 LLMCompiler 比较。实验遵循 ReAct 的设置 (Yao et al., 2022)，采用相同的 Wikipedia 搜索工具，供 LLM 查询信息。由于 lookup 工具与本问题设置无关，我们没有使用它。我们尽力优化了 ReAct 和 LLMCompiler 的提示及上下文

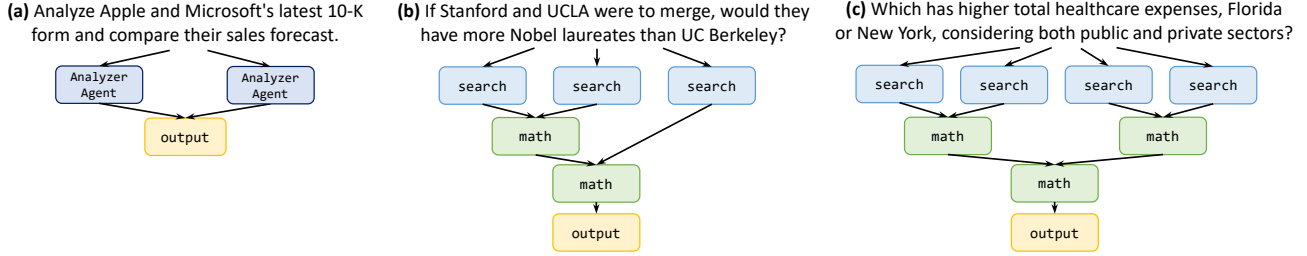


图 3. 具有不同函数调用模式的问题示例及其依赖图。HotpotQA 和 Movie Recommendation 数据集呈现模式 (a); ParallelQA 数据集则包含 (b)、(c) 等多种模式。(a) 需要分析各家公司最新的 10-K 年报。(b) 需要对每所学校进行三次搜索, 再执行一次加法和一次比较。(c) 需要搜索各州每个领域的年度医疗支出, 汇总各州的支出, 再进行比较。图内文字译释: (a) 分析 Apple 与 Microsoft 最新的 10-K 年报, 并比较销售预测; (b) 如果 Stanford 与 UCLA 合并, 其 Nobel 奖获得者是否比 UC Berkeley 更多? (c) 考虑公共与私人部门, Florida 与 New York 哪个州的总医疗支出更高? Analyzer Agent 为分析智能体; search 为搜索; math 为数学计算; output 为输出。

表 1. LLMCompiler 与基线在多个基准上的准确率和时延比较, 包括 HotpotQA、Movie Recommendation、自建数据集 ParallelQA, 以及 24 点游戏。对于 HotpotQA 和 Movie Recommendation, 我们经常观察到循环调用与过早停止 (第 5.1 节)。为了尽量减少这些行为, 我们为 ReAct 加入专门的提示, 将该版本记为 ReAct[†]。没有 [†] 标记的 ReAct 表示未使用这些提示的原始结果。由于循环调用与过早停止使精确测量时延变得困难, 我们不报告原始 ReAct 的时延。

基准	方法	GPT (闭源)			LLaMA-2 70B (开源)		
		准确率 (%)	时延 (秒)	加速比	准确率 (%)	时延 (秒)	加速比
HotpotQA	ReAct	61.52	-	-	54.74	-	-
	ReAct [†]	62.47	7.12	1.00×	54.40	13.44	1.00×
	OAI 并行函数调用	62.05	4.42	1.61×	-	-	-
	LLMCompiler	62.00	3.95	1.80×	57.83	9.58	1.40×
Movie Rec.	ReAct	68.60	-	-	70.00	-	-
	ReAct [†]	72.47	20.47	1.00×	70.60	33.37	1.00×
	OAI 并行函数调用	77.00	7.42	2.76×	-	-	-
	LLMCompiler	77.13	5.47	3.74×	77.80	11.83	2.82×
ParallelQA	ReAct	89.09	35.90	1.00×	59.59	15.47	1.00×
	OAI 并行函数调用	87.32	19.29	1.86×	-	-	-
	LLMCompiler	89.38	16.69	2.15×	68.14	26.20	2.27×
Game of 24	Tree-of-Thoughts	74.00	241.2	1.00×	30.00	952.06	1.00×
	LLMCompiler	75.33	83.6	2.89×	32.00	456.02	2.09×

表 2. HotpotQA、Movie Recommendation 和自建数据集 ParallelQA 的输入、输出 Token 用量, 以及估算成本。成本按各基准所用 GPT 模型的价格表计算。

基准	方法	Token 数		成本 (\$/1k)	成本降低倍数
		输入	输出		
HotpotQA	ReAct	2900	120	5.00	1.00×
	OAI Para. Func.	2500	63	2.66	1.87×
	LLMCompiler	1300	80	1.47	3.37×
Movie Rec.	ReAct	20000	230	20.46	1.00×
	OAI Para. Func.	5800	160	6.14	3.33×
	LLMCompiler	2800	115	3.04	6.73×
ParallelQA	ReAct	46000	470	480	1.00×
	OAI Para. Func.	25000	370	260	1.81×
	LLMCompiler	9200	340	103	4.65×

示例。上述数据集上的所有实验均使用 gpt-3.5-turbo (1106 版本)。在 GPT 实验中, 我们额外报告了 OpenAI 并行函数调用功能的结果; 该功能与本工作同期发布。我们还展示 LLMCompiler 如何与开源 LLaMA-2 70B 模型有效结合, 为其提供并行函数调用能力。全部实验测量准确率、端到端时延, 以及输入和输出 Token 用量。实验设置详见附录 D。

准确率与时延。 表 1 报告了 LLMCompiler 相对于 ReAct 的准确率、端到端时延及加速比。首先, 我们发现 ReAct 的准确率始终低于 OpenAI 并行函数调用和 LLMCompiler。ReAct 有两个主要失败模式: (1) 重复生成已经执行过的函数调用, 这一点也在原始 ReAct 论文中被指出 (Yao et al., 2022); (2) 根据不完整的中间结果过早停止。附录 A 详细分析了这两类常见失败如何显著损害 ReAct 的准确率, 以及 LLMCompiler 如何解决这些问题, 使准确率最多提高 7–8%。此外, 我们开展了干预实验, 为 ReAct 加入专门提示, 要求避免重复调用和过早停止。表 1 中的 ReAct[†] 表示使用了这种专门提示的版本。相对于原始 ReAct, 这些提示普遍改善了 ReAct[†] 的准确率。然而, 提示不能完全避免 ReAct 的错误行为, 因此 LLMCompiler 的准确率仍与 ReAct[†] 相当或更高。

相对于 ReAct[†], LLMCompiler 在 HotpotQA 上使用 GPT 和 LLaMA 时分别获得 1.80× 和 1.40× 的明显加速; 在 Movie Recommendation 上使用两个模型时, 分别获得 3.74× 和 2.82× 的加速。我们用 ReAct[†] 的时延与 LLMCompiler 比较, 是因为原始 ReAct 的重复

调用和过早停止会使其时延难以预测，不适合公平比较。相对于 OpenAI 并行函数调用，LLMCompiler 的加速收益最高为 35%；OpenAI 并行函数调用在两个基准上相对于 ReAct 的加速比分别为 $1.61\times$ 和 $2.76\times$ 。¹

成本。 使用 LLM 时的另一项重要考量是成本，它取决于输入和输出 Token 的用量。GPT 实验的成本列于表 2。LLMCompiler 调用 LLM 的次数更少，因此相对于 ReAct 更节省成本。有意思的是，LLMCompiler 的成本效率也优于近期发布的 OpenAI 并行函数调用。这是因为我们的规划器上下文示例较短，只包含计划而不包含观察结果，使 LLMCompiler 规划阶段的提示长度更低（见附录 H）。

5.2. 具有依赖的并行函数调用

前述情况相对简单：只使用一种工具，且所有任务可以彼此独立执行。但类似传统代码块中的代码执行，函数调用也可能具有更复杂的依赖。为了系统评测复杂任务依赖场景下的函数调用规划能力，我们设计了自建基准 ParallelQA。该基准包含非平凡的函数调用模式，涵盖图 3 中 (b)、(c) 等三种不同模式。受到 IfQA 基准 (Yu et al., 2023) 的启发，ParallelQA 包含 113 个示例，涉及不同实体事实属性的数学问题。具体而言，完成任务需要使用两个工具（搜索与数学工具），第二个工具的参数依赖于第一个工具的输出。我们仔细选取了只需 Wikipedia 第一段信息即可回答的问题，从而排除搜索失败引起的错误。有关 ParallelQA 的更多细节见附录 I。

实验设置。 与第 5.1 节相同，我们以 ReAct (Yao et al., 2022) 为主要基线。ReAct 和 LLMCompiler 均使用两个工具：(1) search，与第 5.1 节的搜索工具相同；(2) math，用于求解数学问题。math 工具受到 Langchain (Langchain) 的 LLMChain 启发：它用 LLM 智能体解释输入查询，再以适当公式调用 numexpr 函数。这样，math 链能够处理以数学式或自然语言表述的多种数学问题。实验设置详见附录 D。

准确率与时延。 如表 1 的 ParallelQA 行所示，LLMCompiler 避免顺序执行依赖图，使用 gpt-4-turbo 和 LLaMA-2 70B 时得到最终答案的平均加速比分别为 $2.15\times$ 和 $2.27\times$ 。除时延改善外，使用 LLaMA-2 时，LLMCompiler 的准确率也高于 ReAct，原因与第 5.1 节的讨论相同。特别是在 LLaMA-2 实验中，LLMCompiler 的准确率提高了约 9%；我们发现 ReAct 在 $\sim 20\%$ 的示例上发生重复函数调用，这与附录 A 的准确率分析一致。附录 B 还全面分析了 LLMCompiler 的失败案例：规划器失败很少，说明 LLMCompiler 能有效地将问题分解为具有复杂依赖的多任务。²

¹ 由于 OpenAI 尚未公开其函数调用机制的细节，我们无法确定这一差异的原因。一种猜测是，验证函数名和参数名、将它们转换为系统提示，可能引入额外开销。不过，在连续多天的多次实验中，我们观察到了稳定的趋势。

² 译注：原表中 ParallelQA 的 LLaMA-2 基线时延为 15.47 秒，本方法为 26.20 秒，却报告 2.27 倍加速，三者不相容。

成本。 与第 5.1 节类似，LLMCompiler 显著降低成本：相对于 ReAct 和 OpenAI 并行函数调用分别减少 $4.65\times$ 和 $2.57\times$ ，如表 2 所示。这一效率来自更少的 LLM 调用次数。OpenAI 并行函数调用同样减少了调用次数，但它只能规划当下即可并行的任务，不能一次规划整个依赖图。例如，在图 3 的 (c) 中，OpenAI 的方法需要三次独立的 LLM 调用，分别处理初始搜索任务、随后的数学任务和最后一个数学任务。LLMCompiler 只需一次 LLM 调用即可同时规划全部任务。

5.3. 带重新规划的并行函数调用

前几节讨论的是能够静态确定依赖图的情形。但有些情形需要根据中间观察结果动态构建依赖图。这里以 24 点游戏为例，考察 (Yao et al., 2023b) 提出的 Tree-of-Thoughts (ToT) 策略这一动态方法。24 点游戏要求使用给定的四个数及基本算术运算得到 24。例如，给定 2、4、4、7，可以得到 $4 \times (7 - 4) \times 2 = 24$ 。ToT 通过两个迭代的 LLM 过程求解：(i) 思路提议器选择两个数并执行一个运算，生成候选部分解（例如从 2、4、4、7 计算 $7 - 4$ ，得到 2、3、7）；(ii) 状态评估器判断各候选的潜力。随后只将有希望的候选送入下一轮思路提议和状态评估，直到得到 24。24 点游戏基准及 ToT 策略详见附录 J。

ToT 显著改善了 24 点游戏的求解效果，但它顺序地对状态树进行广度优先搜索，耗时较长。LLMCompiler 让思路提议器及后续可行性评估器并行执行，提供类似并行束搜索的更快方案。

实验设置。 虽然 LLMCompiler 能够降低时延，但无法用一个静态图求解此问题：在确定上一轮状态评估器选出的候选之前，规划器不能规划下一轮思路提议。因此，规划器每次只能规划一个迭代。为此，我们使用 LLMCompiler 的重新规划能力。具体而言，LLMCompiler 使用三个工具：thought_proposer 和 state_evaluator 都是由原始 ToT 框架改造的 LLM；top_k_select 根据 state_evaluator 的评估，从 thought_proposer 的输出中选取前 k 个候选。所有工具执行完毕后，如果没有提议达到 24，LLMCompiler 可以决定“重新规划”，让规划器使用上一轮 top_k_select 筛出的状态制定新计划。这样，LLMCompiler 可以在每轮动态生成计划，处理必须依据先前计划结果反复重新规划的复杂任务。

我们使用 100 个不同的 24 点游戏实例评测 LLMCompiler。对于每个问题，只有当输出运算合法、结果为 24、且每个给定数字恰好使用一次时，才视为成功。实验设置的更多细节见附录 D。

成功率与时延。 表 1 的最后两行比较了 LLMCompiler 与 (Yao et al., 2023b) 中基线在 24 点游戏上的时延和成功率。使用 gpt-4 时，LLMCompiler 获得 $2.89\times$

本译文保留原数值；该加速比在使用前应向作者或原始结果核实。

的时延改善，成功率也略有提高。使用 LLaMA-2 时，LLMCompiler 获得 $2.01\times$ 的时延改善，同样没有降低成功率。这些结果既展示了保持质量的显著时延下降，也展示了 LLMCompiler 利用重新规划求解复杂问题的能力。³

5.4. 应用：交互式决策任务中的 LLMCompiler

本节通过 WebShop (Yao et al., 2023a) 基准展示 LLMCompiler 有效探索语言交互环境的能力。如 (Shinn et al., 2023; Yao et al., 2022; 2023a) 所述，WebShop 具有较强的多样性，需要充分探索才能购买最合适的商品。近期工作提出了较先进的探索策略，并取得了有希望的结果 (Ma et al., 2023; Zhou et al., 2023a)，但大多依赖顺序执行的大规模树搜索，造成明显时延开销。LLMCompiler 借助并行函数调用，实现既有效又高效的探索策略。它允许更广泛地探索环境中的商品，从而比 ReAct 获得更高的成功率；同时，这些探索可以并行，相对于顺序探索的基线最多获得 $101.7\times$ 的加速。

实验设置。 我们使用 500 条 WebShop 指令，将 LLMCompiler 与三个基线比较：ReAct (Yao et al., 2022)、LATS (Zhou et al., 2023a) 和 LASER (Ma et al., 2023)。指标包括成功率、平均得分和时延。WebShop 环境与基线方法的更多细节见附录 K。本实验为 LLMCompiler 配备 search 和 explore 两个工具。search 让模型生成并提交查询，通常从 WebShop 环境返回十个商品。explore 随后点击每个检索到的商品链接，获取可用选项、价格、属性和功能信息。最后，LLMCompiler 根据收集到的信息，决定购买最符合输入指令的商品。实验细节见附录 D。

性能与时延。 如表 3 所示，本方法显著优于各个基线。使用 gpt-3.5-turbo 时，LLMCompiler 相对于 ReAct 和 LATS 的成功率分别提高 28.4% 和 6%；使用 gpt-4 时，相对于 ReAct 和 LASER 分别提高 20.4% 和 5.6%。在时延方面，LLMCompiler 相对于 LATS 和 LASER 分别获得 $101.7\times$ 和 $2.69\times$ 的加速。我们也发现，在该基准上 LLMCompiler 的执行略慢于 ReAct，主要原因是规划器开销；不过，成功率的收益远大于这部分较小的时延代价。

我们进一步分析 LLMCompiler 相对于 ReAct 获得更高成功率与得分的原因。观察发现，ReAct 智能体倾向于在信息不充分时作出决定；当它没有充分了解商品的功能和可选项时，就可能发生这种情况。文献 (Shinn et al., 2023) 也指出：如果不探索环境中的更多商品，智能体很难区分看起来相似的选项，最终作出错误决定。LLMCompiler 则进一步探索，访问 search 找出的全部十个商品并获取各自的相关信息。我们发现，有效的搜索策略对于 WebShop 这样的决策任务至关重要。

³译注：本段的 LLaMA-2 加速比原文写作 2.01，而主结果表写作 2.09；由表中的 $952.06/456.02$ 计算约为 2.09。本译文分别保留原文中的两处数值。

表 3. WebShop 的性能和时延分析。我们使用 gpt-4 和 gpt-3.5-turbo 两种模型评测 LLMCompiler，并与 ReAct、LATS 和 LASER 三个基线比较。成功率与平均得分均以百分数报告。ReAct 的成功率和平均得分由我们复现，LATS 和 LASER 的对应结果来自各自论文。N 表示用于评测的示例数量。

模型	方法	成功率	得分	时延 (秒)	N
gpt-3.5-turbo	ReAct	19.8	54.2	5.98	500
	LATS	38.0	75.9	1066	50
	LLMCompiler	44.0	72.8	10.72	50
	LLMCompiler	48.2	74.2	10.48	500
gpt-4-0613	ReAct	35.2	58.8	19.90	500
	LASER	50.0	75.6	72.16	500
	LLMCompiler	55.6	77.1	26.73	500

LATS 较高的性能同样可以用其探索方式解释。该框架在 WebShop 的状态与动作空间中执行暴力搜索，在最终购买前探索多达 30 条轨迹。虽然这为决策提供了更丰富的信息，却使端到端执行慢得难以接受。

本方法 LLMCompiler 的平均得分比 LASER 高 1.5。与 LATS 相比，得分差异位于本方法的标准差范围内。使用 gpt-3.5-turbo 时，LLMCompiler 的平均得分及其标准差为 72.8 ± 4.01 。尽管性能差异较小，本方法仍获得显著的执行加速：相对于 LATS 为 $101.7\times$ ，相对于 LASER 为 $2.69\times$ 。

6. 结论

现有使用 LLM 调用多个函数的方法依赖顺序执行和动态推理，因此在时延、成本和准确率方面存在效率问题。为解决这些问题，我们提出 LLMCompiler：一个受到编译器启发的框架，为 LLaMA-2 等开源模型及 OpenAI GPT 系列等不同 LLM 提供高效的并行函数调用。它将用户输入分解为具有明确相互依赖关系的任务，并通过规划器、任务获取单元和执行器并发运行这些任务。LLMCompiler 显著改善了时延（最高 $3.7\times$ ）、成本效率（最高 $6.7\times$ ）和准确率（最高 $\sim 9\%$ ）；其时延收益甚至优于 OpenAI 的并行函数调用功能。我们期待未来工作基于 LLMCompiler，进一步改善 LLM 执行复杂、大规模任务的能力与效率，推动基于 LLM 的应用发展。

影响声明

本论文旨在推动机器学习领域的发展。虽然本工作可能带来多种社会影响，但我们没有发现其中哪一种尤其值得单独强调。

致谢

感谢 Minwoo Kang 提供宝贵反馈，感谢 Furiosa 团队的慷慨支持。感谢 Microsoft 通过 Accelerating Foundation Model Research 项目提供支持，特别感谢 Sean Kuno。还感谢 Google Cloud、Google TRC 团队，尤其是 Jonathan Caton 和 David Patterson 教授。Keutzer 教授的实验室得到 Intel 公司、Intel One-API、Intel VLAB

团队、Intel One-API 卓越中心以及 BDD 和 BAIR 的资助。感谢 Samsung 的支持，特别是 Dongkyun Kim 和 David Thorsley。感谢 Ellick Chan、Saurabh Tangri、Andres Rodriguez 和 Kittur Ganesh 的大力支持。Sehoon Kim 与 Suhong Moon 感谢韩国高等教育财团的支持。Amir Gholami 得到 Samsung SAIT 的资助。Michael W. Mahoney 感谢 J. P. Morgan Chase 教师研究奖，以及 DOE、NSF 和 IARPA 的支持。本文结论不一定代表资助方的立场或政策，也不应据此推断其正式认可。

参考文献

- <https://huggingface.co/text-generation-inference>.
- <https://github.com/nvidia/tensorrt-llm>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021.
- Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., Gajda, J., Lehmann, T., Podstawski, M., Niewiadomski, H., Nyczyk, P., and Hoefer, T. Graph of thoughts: Solving elaborate problems with large language models, 2023.
- Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling, 2023a.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. 2021.
- Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks, 2023b.
- Dettmers, T., Svirschevski, R., Egiazarian, V., Kuznedelev, D., Frantar, E., Ashkboos, S., Borzunov, A., Hoefer, T., and Alistarh, D. Spqr: A sparse-quantized representation for near-lossless llm weight compression, 2023.
- Frantar, E. and Alistarh, D. Sparsegpt: Massive language models can be accurately pruned in one-shot, 2023.
- Frantar, E., Ashkboos, S., Hoefer, T., and Alistarh, D. GPTQ: Accurate post-training compression for generative pretrained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- Gao, L., Madaan, A., Zhou, S., Alon, U., Liu, P., Yang, Y., Callan, J., and Neubig, G. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022.
- Hao, S., Gu, Y., Ma, H., Hong, J. J., Wang, Z., Wang, D. Z., and Hu, Z. Reasoning with language model is planning with world model, 2023.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., and Steinhardt, J. Measuring coding challenge competence with apps. *NeurIPS*, 2021a.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021b.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *NeurIPS*, 2021c.
- Houlsby, N., Giurghi, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pp. 2790–2799. PMLR, 2019.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Karpathy, A. Intro to large language models, 2023.
- Khot, T., Trivedi, H., Finlayson, M., Fu, Y., Richardson, K., Clark, P., and Sabharwal, A. Decomposed prompting: A modular approach for solving complex tasks. In *The Eleventh International Conference on Learning Representations*, 2023.

- Kim, S., Hooper, C., Gholami, A., Dong, Z., Li, X., Shen, S., Mahoney, M. W., and Keutzer, K. Squeezellm: Dense-and-sparse quantization, 2023.
- Kim, S., Mangalam, K., Moon, S., Malik, J., Mahoney, M. W., Gholami, A., and Keutzer, K. Speculative decoding with big little decoder, 2024.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners, 2023.
- Kwon, W., Kim, S., Mahoney, M. W., Hassoun, J., Keutzer, K., and Gholami, A. A fast post-training pruning framework for transformers, 2022.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- Langchain. <https://github.com/langchain-ai/langchain>.
- Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning, 2021.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding, 2023.
- Liang, Y., Wu, C., Song, T., Wu, W., Xia, Y., Liu, Y., Ou, Y., Lu, S., Ji, L., Mao, S., Wang, Y., Shou, L., Gong, M., and Duan, N. Taskmatrix.ai: Completing tasks by connecting foundation models with millions of apis, 2023.
- Lin, J., Tang, J., Tang, H., Yang, S., Dang, X., Gan, C., and Han, S. Awq: Activation-aware weight quantization for llm compression and acceleration, 2023.
- Liu, J. LlamaIndex, 11 2022. URL https://github.com/jerryjliu/llama_index.
- Ma, K., Zhang, H., Wang, H., Pan, X., and Yu, D. Laser: Llm agent with state-space exploration for web navigation, 2023.
- Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegrefe, S., Alon, U., Dziri, N., Prabhume, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. Self-refine: Iterative refinement with self-feedback, 2023.
- Ning, X., Lin, Z., Zhou, Z., Wang, Z., Yang, H., and Wang, Y. Skeleton-of-thought: Large language models can do parallel decoding, 2023.
- OpenAI. Gpt-4 technical report, 2023.
- OpenAI. New models and developer products announced at devday, 2023.
- Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., and Gonzalez, J. E. Memgpt: Towards llms as operating systems, 2023.
- Patel, P., Mishra, S., Parmar, M., and Baral, C. Is a question decomposition unit all we need? 2022.
- Patil, S. G., Zhang, T., Wang, X., and Gonzalez, J. E. Gorilla: Large language model connected with massive apis, 2023.
- Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., and Lewis, M. Measuring and narrowing the compositionality gap in language models, 2023.
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.
- Ruan, J., Chen, Y., Zhang, B., Xu, Z., Bao, T., Du, G., Shi, S., Mao, H., Zeng, X., and Zhao, R. Tptu: Task planning and tool usage of large language model-based ai agents. *arXiv preprint arXiv:2308.03427*, 2023a.
- Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T. Identifying the risks of lm agents with an lm-emulated sandbox. *arXiv preprint arXiv:2309.15817*, 2023b.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.
- Shen, Y., Song, K., Tan, X., Li, D., Lu, W., and Zhuang, Y. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face, 2023.
- Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., and Yao, S. Reflexion: Language agents with verbal reinforcement learning, 2023.
- Song, Y., Xiong, W., Zhu, D., Wu, W., Qian, H., Song, M., Huang, H., Li, C., Wang, K., Yao, R., Tian, Y., and Li, S. Restgpt: Connecting large language models with real-world restful apis, 2023.

- Srivastava, A., Rastogi, A., Rao, A., Shoeb, A. A. M., Abid, A., Fisch, A., Brown, A. R., Santoro, A., Gupta, A., Garriga-Alonso, A., et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *arXiv preprint arXiv:2206.04615*, 2022.
- Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. Cognitive architectures for language agents, 2023.
- Surís, D., Menon, S., and Vondrick, C. Vipergpt: Visual inference via python execution for reasoning, 2023.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardaş, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023.
- Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R. K.-W., and Lim, E.-P. Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. *arXiv preprint arXiv:2305.04091*, 2023a.
- Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., and Zhou, D. Self-consistency improves chain of thought reasoning in language models, 2023b.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. volume 35, pp. 24824–24837, 2022.
- Wolfson, T., Geva, M., Gupta, A., Gardner, M., Goldberg, Y., Deutch, D., and Berant, J. Break it down: A question understanding benchmark. *Transactions of the Association for Computational Linguistics*, 2020.
- Xu, B., Peng, Z., Lei, B., Mukherjee, S., Liu, Y., and Xu, D. Rewoo: Decoupling reasoning from observations for efficient augmented language models, 2023.
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., and Manning, C. D. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Yang, Z., Dong, L., Du, X., Cheng, H., Cambria, E., Liu, X., Gao, J., and Wei, F. Language models as inductive reasoners, 2022.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Yao, S., Chen, H., Yang, J., and Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023a.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., and Narasimhan, K. Tree of Thoughts: Deliberate problem solving with large language models, 2023b.
- Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pp. 521–538, 2022.
- Yu, W., Jiang, M., Clark, P., and Sabharwal, A. Ifqa: A dataset for open-domain question answering under counterfactual presuppositions, 2023.
- Zheng, H. S., Mishra, S., Chen, X., Cheng, H.-T., Chi, E. H., Le, Q. V., and Zhou, D. Take a step back: Evoking reasoning via abstraction in large language models, 2023.
- Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., and Wang, Y.-X. Language agent tree search unifies reasoning acting and planning in language models, 2023a.
- Zhou, D., Schärli, N., Hou, L., Wei, J., Scales, N., Wang, X., Schuurmans, D., Cui, C., Bousquet, O., Le, Q. V., and Chi, E. H. Least-to-most prompting enables complex reasoning in large language models. In *The Eleventh International Conference on Learning Representations*, 2023b.

A. 准确率分析：ReAct 与 LLMCompiler

本节详细比较 ReAct 与 LLMCompiler 的准确率，重点分析 ReAct 中常见的两类失败：(i) 过早停止；(ii) 重复函数调用。我们还展示：这些失败会损害 ReAct 的准确率，而 LLMCompiler 能有效解决这些问题，从而提高框架的准确率。我们分析两个具体场景：在使用 GPT 的 Movie Recommendation 评测中，ReAct 经常过早停止，其准确率显著低于 LLMCompiler（表 1 中为 68.60 与 77.13）；在使用 LLaMA-2 70B 的 HotpotQA 评测中，ReAct 的重复调用使准确率明显低于 LLMCompiler（表 1 中为 70.00 与 77.80）。⁴

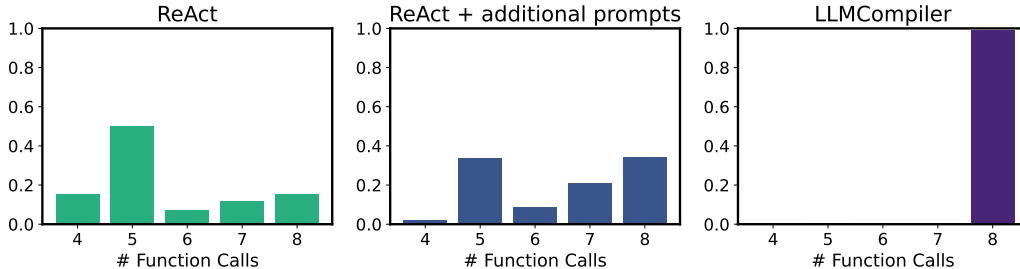


图 A.1. 在 Movie Recommendation 基准上，ReAct（左）、加入避免过早停止的专门提示的 ReAct（中，对应表 1 中的 ReAct[†]），以及 LLMCompiler（右）的函数调用次数分布。LLMCompiler（右）始终完成全部八部电影的搜索；ReAct（左）则经常提前退出，约 85% 的示例过早停止。虽然定制提示使 ReAct 的直方图向更多调用次数移动（中），却仍不能保证对全部电影完成搜索。本实验使用 gpt-3.5-turbo。图内文字译释：横轴为函数调用次数，纵轴为示例比例；ReAct + additional prompts 为加入额外提示的 ReAct，其余图例为对应框架名称。

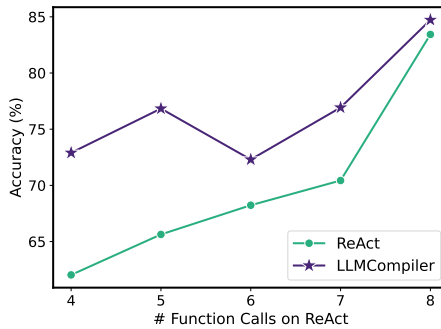


图 A.2. 按 ReAct 的函数调用次数分组的示例，在 ReAct 和 LLMCompiler 上各自的 Movie Recommendation 准确率。图中显示，对于 ReAct，调用次数减少与准确率下降相关，说明过早退出导致准确率降低。LLMCompiler 在作出决定前确保搜索全部八部电影，因此在相同示例上获得更高、更稳定的准确率。本实验使用 gpt-3.5-turbo，结果为三次不同运行的平均值。图内文字译释：横轴为 ReAct 的函数调用次数；纵轴 Accuracy (%) 为准确率；绿色与紫色曲线分别对应 ReAct 和 LLMCompiler。

ReAct 的过早停止。 ReAct 经常过早停止函数调用，并依据不完整的信息作出决定。Movie Recommendation 基准清楚地展示了这一问题：ReAct 在返回最终答案前，搜索的电影数往往少于所需的八部。图 A.1（左）显示了使用 GPT 的 ReAct 在该基准上的函数调用次数分布。约 85% 的示例没有完成全部八部电影的搜索就作出决定。相比之下，LLMCompiler（右）几乎所有示例（99%）都完成八部电影的完整搜索。虽然加入避免过早停止的专门提示后，ReAct 的调用次数分布向右移动（图 A.1，中），准确率也从 68.60 提高到 72.47（表 1 中的 ReAct[†]），但这种方案仍不完善。

为了进一步评估过早停止对准确率的损害，我们按 ReAct 的函数调用次数将 Movie Recommendation 示例分组，再用 LLMCompiler 评测相同分组，确保得到全部八部电影的搜索结果。图 A.2 显示，ReAct 的调用次数越少，平均准确率越低（绿色曲线）；相同示例经由 LLMCompiler 处理、完成八部电影的全部搜索后，准确率始终更高（紫色曲线）。这既说明 ReAct 存在无法仅靠提示完全解决的过早退出问题，也说明停止得越早，准确率下降越多，从而造成表 1 中观察到的整体准确率损失。LLMCompiler 能够有效解决这一问题。

⁴译注：原文将第二个场景写为 HotpotQA，却引用了 70.00 与 77.80；主结果表中这两项实际属于 LLaMA-2 的 Movie Recommendation，HotpotQA 对应的是 54.74 与 57.83。这里保留原文并明确指出错配。

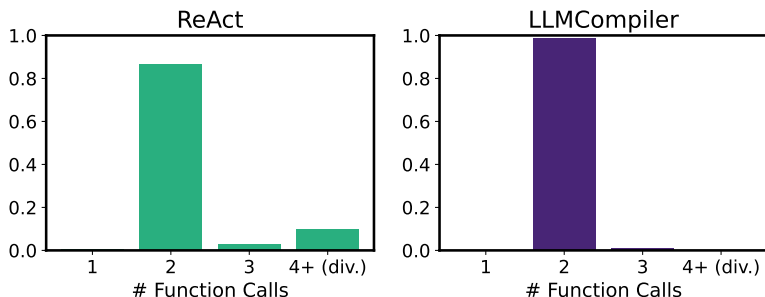


图 A.3. 在 HotpotQA 基准上, ReAct (左) 与 LLMCompiler (右) 的函数调用次数分布。HotpotQA 具有两路可并行模式, 因此 LLMCompiler (右) 始终在两次函数调用内完成任务; ReAct (左) 则在约 10% 的示例上发生重复调用 (>4 次), 导致框架出现发散行为。本实验使用 LLaMA-2 70B。图内文字译释: 横轴为函数调用次数, 纵轴为示例比例; 4+ (div.) 表示四次及以上调用的发散类别; 图例为对应框架名称。

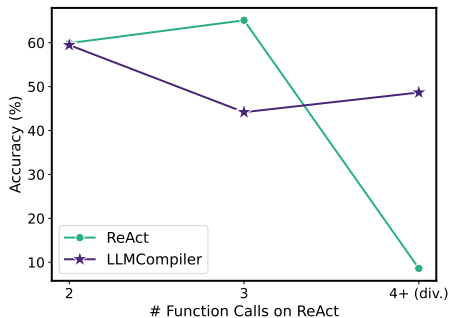


图 A.4. 按 ReAct 的函数调用次数分组的示例, 在 ReAct 和 LLMCompiler 上各自的 HotpotQA 准确率。图中显示, ReAct 的重复调用达到四次或更多时, 会由于无限循环和发散行为导致准确率显著下降。相同示例经由 LLMCompiler 处理、确保每例只进行两次搜索后, 准确率提高到约 50%。本实验使用 LLaMA-2 70B。图内文字译释: 横轴为 ReAct 的函数调用次数; 纵轴为准确率 (百分数); 4+ (div.) 表示四次及以上调用的发散类别。

ReAct 的重复函数调用。 ReAct 的另一类常见失败是重复调用函数, 往往导致无限循环或超过上下文长度限制。该问题在 HotpotQA 上尤其明显: 如果 Wikipedia 搜索没有返回足够的实体信息, ReAct 就会反复调用同一个函数。HotpotQA 本来具有两路可并行性, 但如 图 A.3 所示, 约 10% 的示例在 ReAct 中需要超过四次函数调用, 通常最终进入无限循环或发生发散。LLMCompiler 则对大多数示例只进行两次函数调用。

为了说明重复调用对整体准确率的影响, 我们采用与前述情形相似的准确率分析。图 A.4 按 ReAct 的调用次数对 HotpotQA 示例分组, 并比较这些示例在 ReAct 和 LLMCompiler 上的准确率。分析表明, 在 ReAct 中进行两次调用的示例, 在 LLMCompiler 中保持相同准确率; 而在 ReAct 中超过四次调用、经常发散的示例, ReAct 的准确率不足 10%。这些示例经由 LLMCompiler 处理时, 由于避免重复调用, 准确率约为 50%。也存在 ReAct 进行三次调用的情况: 首次搜索失败后, 用另一个实体名重新搜索, 额外一次搜索能够提高准确率, 甚至优于 LLMCompiler。这显示了 ReAct 在适应性上的潜在优势, 但此类实例不足全部案例的 3%。

B. LLMCompiler 的失败案例分析

本节定性分析 LLMCompiler 在 ParallelQA 基准上的失败案例, 主要归因于规划器、执行器或最终输出过程。最终输出失败指 LLM 未能利用工具执行所收集、已纳入上下文的观察结果, 为用户给出正确答案。在表 1 报告的 10.6% 总失败 (36 个示例) 中, 规划器、执行器和最终输出过程分别占失败的 8%、64% 和 28%。规划器所占的 8% 失败是 LLMCompiler 特有的: 例如, 将错误的标识符作为后续任务输入, 错误映射输入与输出, 形成错误的 DAG。不过, 提供充分的工具定义和上下文示例后, 规划器错误显著减少, 在整个评测中仅发生三例。这说明 LLM 能将问题分解为具有复杂多任务依赖的结构。

其余 92% 的失败来自执行器和最终输出过程。执行器造成大多数失败 (64%), 常见问题包括 math 工具选择错误属性、错误处理单位换算等。最终输出过程造成 28% 的失败, 主要表现为依据已收集的观察结果得出错误结论, 例如未能从数据中选出最小属性。这些问题并非 LLMCompiler 特有, ReAct 也会出现。由于 LLMCompiler

表 C.1. 规划器启用与禁用流式输出的时延比较。流式输出使任务获取单元在规划器生成每个任务后立即开始执行，因此在不同基准上都改善时延。在 ParallelQA 上，工具执行时间足以有效掩盖规划器的执行时间，因而流式输出的效果尤其明显。

基准	禁用流式 (秒)	启用流式 (秒)	时延加速比
HotpotQA	4.00	3.95	1.01×
Movie Rec.	5.64	5.47	1.03×
ParallelQA	21.72	16.69	1.30×

只向每个工具提供相关上下文，有利于准确提取信息，它在这两类问题上的失败略少于 ReAct。我们认为，优化智能体草稿区的结构，而非简单追加观察结果，还可以进一步减少最终输出失败。

C. 相关工作

本节继续第 2 节的相关工作讨论。

C.1. 工具增强的 LLM

Toolformer (Schick et al., 2023) 是一项值得关注的工作：它构造定制的 LLM 输出，让 LLM 决定函数调用的输入，以及将结果插入到哪个位置。该方法启发了多种工具调用框架 (Liang et al., 2023; Shen et al., 2023)。ReAct (Yao et al., 2022) 让 LLM 通过推理和动作生成与外部环境交互，以改善任务表现。Gorilla (Patil et al., 2023) 提出专门用于函数调用的微调 LLM；ToolLLM (Qin et al., 2023) 和 RestGPT (Song et al., 2023) 将 LLM 扩展到真实世界的 API。此外，OpenAI (OpenAI, 2023) 发布了自己的函数调用能力，使其 LLM 可以返回格式化 JSON 供执行。

D. 实验细节

实验评估两种常见场景：(1) 通过 API 使用闭源模型；(2) 在自建服务框架上使用开源模型。闭源模型采用 OpenAI GPT 系列：HotpotQA 与 Movie Recommendation 使用 gpt-3.5-turbo (1106 版本)，ParallelQA 使用 gpt-4-turbo (1106 版本)，24 点游戏使用 gpt-4 (0613 版本)。HotpotQA、Movie Recommendation 和 ParallelQA 的实验均在 1106 版本发布后的 2023 年 11 月开展；24 点游戏实验在 2023 年 9 月至 10 月的两个月内开展。开源模型采用 LLaMA-2 (Touvron et al., 2023)，通过 vLLM (Kwon et al., 2023) 托管在两块 A100-80GB GPU 上。除 24 点游戏的 `thought_proposer` 和 `state_evaluator` 使用 0.7 的温度外，所有运行均设为零温度。OpenAI 即使在温度为 0 时也会产生随机输出，因此我们运行三次并报告平均准确率。对于 ReAct、OpenAI 并行函数调用和 LLMCompiler，HotpotQA、Movie Recommendation 和 ParallelQA 分别采用 3、1、5 个示例的少样本学习；不同方法使用相同示例，以保证公平比较。24 点游戏的规划器使用两个上下文示例。各方法使用相同的指令提示，仅第 5.1 节的 ReAct[†] 额外加入 ReAct 专用提示。WebShop 实验使用具有 8k 上下文窗口的 gpt-4-0613，以及具有 16k 上下文窗口的 gpt-3.5-turbo。

E. 分析

E.1. 并行加速建模

虽然 LLMCompiler 在多种工作负载上明显降低时延，但对于 N 路并行工作负载，它并未达到 $N \times$ 的时延加速。这主要源于无法并行的规划器和最终回答过程的开销。在 Movie Recommendation 实验中，规划器与回答过程的平均开销分别为 1.88 秒和 1.62 秒，合计已经超过表 1 中 LLMCompiler 总时延的一半。另一类开销是并行任务汇合时的慢任务效应：最慢 `search` 的平均时延为 1.13 秒，几乎是所有任务平均时延 0.61 秒的 $2 \times$ 。下面对 ReAct、LLMCompiler，以及启用流式机制的 LLMCompiler 建立分析性时延模型，并分析可以达到的加速比。

本节聚焦完全可并行工作负载 (图 3 的模式 (a))，以更清楚地理解各个组件对潜在时延收益的影响。为精确分析时延，我们考察图 2 中的三个关键组件：规划器、任务获取单元和执行器。假设规划器生成 N 个不同的待执行任务。令 P_i 表示规划器与第 i 个原子任务对应的输出；每个 P_i 是一个特定原子任务的蓝图，将该任务记为 E_i 。执行 E_i 时，用对应工具进行一次函数调用。系统中每个单元的时延函数描述特定操作耗费的时间。规划器的时延记为 $T_P(P_i)$ ，即生成计划 P_i 的时间。类似地，执行器的时延 $T_E(E_i)$ 表示完成任务 E_i 所需的时间。

本节忽略任务制定单元的时延，因为其开销可以忽略。⁵我们主要比较 ReAct (Yao et al., 2022) 与 LLMCompiler 的时延模型。

首先，将 ReAct 的总时延表示为：

$$T^R = \sum_{i=1}^N (T_P^R(P_i) + T_E(E_i)). \quad (1)$$

上标 R 表示 ReAct。在 ReAct 智能体系统中，通常先生成思考，再生成动作，并通过对应工具的函数调用获取观察结果。思考与动作的生成共同视为生成 P_i 的过程。规划器的时延带有表示 ReAct 的上标，而执行器的时延没有这种上标，因为 ReAct 与 LLMCompiler 使用的函数调用和工具执行相同。

对于 LLMCompiler，所有可并行任务并发处理，总时延由这些任务中最慢者决定。因此，LLMCompiler 的时延模型可表示为：

$$T^C = \sum_{i=1}^N T_P^C(P_i) + \max_{k=1,\dots,N} T_E(E_k). \quad (2)$$

该式包含全部规划时间之和，以及执行时间最长任务的时间，反映了系统的并行执行方式。

进一步，如果规划器以流式方式输出依赖图，则修改后的时延模型可表示为：

$$T^{SC} = \sum_{i=1}^N T_P^C(P_i) + T_E(E_N). \quad (3)$$

原文指出 $T^{SC} \leq T^C$ 。这意味着流式机制能更高效地处理任务依赖，有可能降低总时延。⁶

评估 LLMCompiler 相对于 ReAct 可达到的加速收益时，定义加速指标 γ 如下：

$$\gamma = \frac{T^R}{T^C} = \frac{\sum_{i=1}^N (T_P^R(P_i) + T_E(E_i))}{\sum_{i=1}^N T_P^C(P_i) + \max_{k=1,\dots,N} T_E(E_k)}. \quad (4)$$

该比值同时考虑规划和执行时延，表示 LLMCompiler 相对于 ReAct 的效率。

为了估计加速比的上界 $\gamma_{\max}$ ，假设执行器时延 $T_E(E_i)$ 远大于规划时延 $T_P(P_i)$ ，且所有任务的执行时延相同。在这一假设下，上界为：

$$\gamma_{\max} \approx \frac{\sum_{i=1}^N T_E(E_i)}{\max_{k=1,\dots,N} T_E(E_k)} = N, \quad (5)$$

即理论最大加速比 $\gamma_{\max}$ 等于任务数 N 。

另一方面，当规划时延占主导时，加速比 γ 达到下界。由于 ReAct 与 LLMCompiler 的规划时延通常相近，最小加速比近似为：

$$\gamma_{\min} \approx \frac{\sum_{i=1}^N T_P^R(P_i)}{\sum_{i=1}^N T_P^C(P_i)} \approx 1. \quad (6)$$

由上述观察可知，要使 LLMCompiler 获得显著时延收益，关键在于：(i) 降低规划器开销；(ii) 尽量减少慢任务。

E.2. 时延与可并行任务数量的关系

图 E.5 进一步给出了 ParallelQA 上的时延分解，将端到端时延表示为并行任务数量的函数。这在高性能计算中通常称为弱扩展性：理想情况下，任务数量增加时，时延保持不变。ReAct 的时延随任务数成比例增长，符合其

⁵译注：原文此处写作 Task Formulation Unit，正文所定义的组件名称为 Task Fetching Unit；这里指同一任务获取单元。

⁶译注：上式以最后生成任务的完成时刻表示整个流式执行的完成时刻，隐含该任务最后完成、且其他任务执行已被规划过程掩盖的条件。一般情况下，需要对所有任务的“该任务可启动时刻加执行时间”取最大值；不能无条件用最后一个任务的执行时间替代这一最大值。原式及不等式均保留。

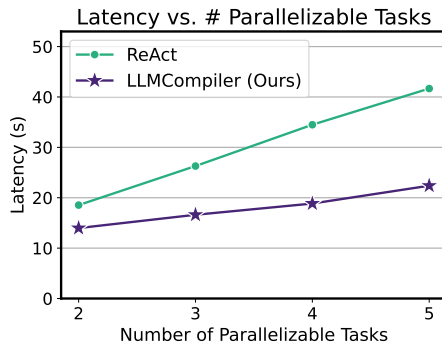


图 E.5. 按最大可并行任务数分组的 ParallelQA 基准时延。图内文字译释：标题为“时延与可并行任务数的关系”；横轴为可并行任务数；纵轴为时延（秒）；LLMCompiler (Ours) 为本方法 LLMCompiler，另一条曲线为 ReAct。

表 E.2. LLMCompiler 与 ReAct 在 HotpotQA 的 bridge 基准上的准确率和时延比较。ReAct[†] 表示加入了额外提示以减少循环调用和过早停止的 ReAct，与表 1 的设置类似。

方法	准确率 (%)	时延 (秒)
ReAct	22.7	7.07
ReAct [†]	23.1	6.42
LLMCompiler	26.3	4.70

表 E.3. LLMCompiler 与其他框架的定性比较，包括 ReAct (Yao et al., 2022)、TPTU (SA 表示 Sequential Agent, 即顺序智能体; OA 表示 One-step Agent, 即单步智能体) (Ruan et al., 2023a)、ViperGPT (Surís et al., 2023) 和 HuggingGPT (Shen et al., 2023)。

方法	规划	重新规划	并行执行	适用领域
ReAct	X	-	X	全部
TPTU-SA	X	-	X	全部
TPTU-OA	O	X	X	全部
ViperGPT	O	X	X	有限
HuggingGPT	O	X	O	有限
LLMCompiler	O	O	O	全部

顺序执行方式。LLMCompiler 则能在可能时并行运行多个函数调用，因此时延增长更慢。LLMCompiler 的端到端时延仍略有增长，是因为规划器必须先生成计划，而这一开销不能并行。附录 E.1 对此作了进一步分析。

E.3. HotpotQA bridge 基准上的补充实验

在第 5.1 节的主要实验中，我们使用 HotpotQA 的 comparison 基准，展示 LLMCompiler 高效执行两路可并行工作负载的能力。该基准的另一部分称为 bridge，涉及顺序任务，例如“在电影 Kiss and Tell 中饰演 Corliss Archer 的女性曾担任什么政府职位？”。LLMCompiler 也可以利用重新规划能力处理 bridge 基准：对于上述例子，先搜索谁在 Kiss and Tell 中饰演 Corliss Archer，再通过重新规划搜索这位女性曾担任的政府职位。

与 comparison 基准的实验相同，表 E.2 将 LLMCompiler 与 ReAct，以及加入避免重复调用和过早停止提示的 ReAct (ReAct[†]) 比较。我们观察到准确率分别提高 4 和 3%；这来自 ReAct 的重复函数调用问题，即使加入额外提示 (ReAct[†])，仍有 5% 的示例因此失败。重复调用也造成 ReAct 的时延略高于本方法。这一实验说明，LLMCompiler 对并行和顺序工作负载都能提供高效、准确的函数调用。

F. 关于相关工作的补充讨论

TPTU (Ruan et al., 2023a)、HuggingGPT (Shen et al., 2023) 和 ViperGPT (Surís et al., 2023) 提出了端到端的先规划再求解框架。本节从以下方面讨论 LLMCompiler 与其他框架的区别：(i) 规划与重新规划能力；(ii) 并行执行；(iii) 更广泛的问题领域。概览见表 E.3。

并行执行：并行执行是 LLMCompiler 的关键功能，使函数调用和任务完成更加高效。TPTU 的单步智能体 (TPTU-OA) 虽然包含规划，却只把用户输入分解为函数及参数的序列，没有表示它们的相互依赖，因此不支持

表 F.4. 使用 gpt-3.5-turbo 时, LLMCompiler 与 ReAct、TPTU (SA 表示 Sequential Agent, 即顺序智能体; OA 表示 One-step Agent, 即单步智能体) 在 HotpotQA comparison 基准上的准确率及加速比比较。ReAct[†] 与 TPTU-SA[†] 分别表示加入额外提示以减少循环调用和过早停止的版本, 与表 1 的设置类似。

方法	准确率 (%)	加速比
ReAct	61.52	-
ReAct [†]	62.47	1×
TPTU-SA	34.16	-
TPTU-SA [†]	44.59	1.09×
TPTU-OA	57.50	1.35×
LLMCompiler	62.00	1.51×

并行函数调用。ViperGPT 生成 Python 代码, 但标准 Python 解释器本身不提供并行执行支持, 因此没有专门的并行处理引擎时, ViperGPT 自身不能并行执行。HuggingGPT 支持并行执行, 但严格面向 HuggingFace 中的模型, 难以应用于 LLMCompiler 所支持的广泛问题和领域。

规划与重新规划: TPTU 的顺序智能体 (TPTU-SA) 与 ReAct (Yao et al., 2022) 一样, 是每轮执行一个动作的迭代框架。TPTU-OA、HuggingGPT 和 ViperGPT 都采用规划, 在执行前制定多个动作, 但缺乏重新规划能力。LLMCompiler 则在已有计划不足以回答用户时, 通过重新规划生成新的一组任务。这使 LLMCompiler 无须引入复杂分支逻辑, 就能依据事先未知的中间结果调整计划, 扩展可处理的问题范围。

问题领域: ViperGPT 和 HuggingGPT 分别通过 Python 代码生成及 HuggingFace 中的模型面向视觉任务, 在这些特定领域展现出较大潜力。LLMCompiler 则提供一个通用框架, 在广泛的问题领域实现高效、准确的函数调用。

F.1. LLMCompiler 与 TPTU 的定量比较

表 F.4 进一步报告了 LLMCompiler 相对于 TPTU-SA 和 TPTU-OA 的准确率与时延加速比。由于 TPTU 没有公开官方实现, 我们依据原文中的提示实现了 TPTU-SA 和 TPTU-OA。表中结果清楚展示了 LLMCompiler 对二者的时延与准确率优势。TPTU 经常重复函数调用, 因此与 TPTU-SA 相比, LLMCompiler 的准确率显著提高。即使优化提示 (TPTU-SA[†]), 这一问题也没有完全解决, 仍有 ~15% 的示例因重复调用失败。相对于 TPTU-SA 和 TPTU-OA, LLMCompiler 也通过并行任务执行降低了时延。整体结果与对 ReAct、OpenAI 并行函数调用等其他基线的主要实验和分析一致。

G. 配置 LLMCompiler 时由用户提供的示例

LLMCompiler 提供简单接口, 允许用户通过工具定义和可选的规划器上下文示例, 为不同用例定制框架。下面给出设置 Movie Recommendation 和 24 点游戏基准所用的规划器示例提示; 只需少量提示即可完成配置。

G.1. Movie Recommendation 的示例提示

问题: 找到一部与 Mission Impossible、The Silence of the Lambs、American Beauty、Star Wars Episode IV - A New Hope 相似的电影。

选项:

Austin Powers International Man of Mystery
Alesha Popovich and Tugarin the Dragon
In Cold Blood
Rosetta

```
1. search("Mission Impossible")
2. search("The Silence of the Lambs")
3. search("American Beauty")
4. search("Star Wars Episode IV - A New Hope")
5. search("Austin Powers International Man of Mystery")
6. search("Alesha Popovich and Tugarin the Dragon")
```

```

7. search("In Cold Blood")
8. search("Rosetta")
Thought: 我现在可以回答问题了。
9. finish()
###

```

G.2. 24 点游戏的示例提示

```

问题: "1 2 3 4", state_list: [""]
$1 = thought_proposer("1 2 3 4", "")
$2 = state_evaluator("1 2 3 4", "$1")
$3 = top_k_select("1 2 3 4", ["$1"], ["$2"])
$4 = finish()
###
问 题: "1 2 3 4", state_list: ["1+2=3(left:3 3 4)", "2-1=1(left:1 3 4)", "3-1=2(left:2 2 4)", "4-1=3(left:2 3 3)", "2*1=2(left:2 3 4)"]a
$1 = thought_proposer("1 2 3 4", "1+2=3(left:3 3 4)")
$2 = thought_proposer("1 2 3 4", "2-1=1(left:1 3 4)")
$3 = thought_proposer("1 2 3 4", "3-1=2(left:2 2 4)")
$4 = thought_proposer("1 2 3 4", "4-1=3(left:2 3 3)")
$5 = thought_proposer("1 2 3 4", "2*1=2(left:2 3 4)")
$6 = state_evaluator("1 2 3 4", "$1")
$7 = state_evaluator("1 2 3 4", "$2")
$8 = state_evaluator("1 2 3 4", "$3")
$9 = state_evaluator("1 2 3 4", "$4")
$10 = state_evaluator("1 2 3 4", "$5")
$11 = top_k_select("1 2 3 4", ["$1", "$2", "$3", "$4", "$5"], ["$6", "$7", "$8", "$9", "$10"])
$12 = finish()
###

```

^a译注: state_list 是状态列表, 字符串中的 left 表示运算后剩余的数字; 函数名、变量标识符和状态字符串属于示例代码, 保留原样。

H. LLMCompiler 规划器的预定义提示

LLMCompiler 规划器的预定义提示给出具体指令, 指导它分解任务、生成依赖图, 同时保证相关语法格式正确。提示包括以下规则: 每个任务单独占一行, 以数字标识符开头, 并用 \$ 符号表示中间变量。

- 上述每个动作都包含输入、输出类型及描述。
- 你必须严格遵守每个动作的输入和输出类型。
- 动作描述中包含使用指南。使用动作时, 你必须严格遵循这些指南。
- 计划中的每个动作必须严格属于上述类型之一。每个动作遵循 Python 的约定。
- 每个动作必须具有唯一且严格递增的 ID。
- 动作的输入可以是常量, 也可以是先前动作的输出。后一种情况下, 使用 \$id 格式, 表示将输出作为输入的先前动作 ID。
- 确保计划最大限度地支持并行。
- 只使用所提供的动作类型。如果这些动作不能处理某个查询, 则调用 finish 动作以进行下一步处理。
- 绝不要通过注释 (例如 #) 解释计划。
- 绝不要引入所提供动作之外的新动作。

除用户提供的函数外，规划器还包含一个特殊的、在实现中固定定义的 `finish` 函数。当计划足以回答用户查询，或必须先执行当前计划、无法继续规划（即判断需要重新规划）时，规划器使用该函数。规划器输出 `finish` 后停止生成计划。附录 G 给出了在规划中使用 `finish` 的示例。`finish` 的定义如下，与用户提供的其他函数定义一起作为提示交给规划器。⁷

```
finish():
- 收集并合并先前动作的结果。
- 调用 join 时，会调用一个 LLM 智能体，用于最终回答用户查询，或等待计划执行完毕。
- join 必须始终是计划中的最后一个动作，以下两种情况下会调用它：
  (a) 可以通过汇总各任务输出，确定答案并生成最终回复。
  (b) 在执行计划之前，无法在规划阶段确定答案。
```

I. ParallelQA 基准的构建

受到 IfQA 基准 (Yu et al., 2023) 的启发，自建基准 ParallelQA 包含 113 个示例：对不同实体的事实细节提出数学问题，要求执行以不同方式相互依赖的搜索和数学运算。例如，“如果 Texas 与 Florida 合并为一个州，California 与 Michigan 也合并为一个州，这两个新州中更大的人口密度是多少？”需要四个并行搜索任务，随后执行依赖搜索结果、也可以并行的数学任务。

该基准的主要目标是衡量框架将输入分解为多个任务、进而求出答案的能力。因此，我们从不同领域仔细选取了 56 个实体，其属性可以通过 Wikipedia 搜索获得。通过尽量减少工具执行（即 Wikipedia 搜索）失败，我们使该基准有效评估框架分解问题、规划任务，并依据观察结果得出最终答案的能力。为了涵盖多样的执行模式，我们构造了不同依赖模式：先搜索问题中实体的补充信息，再执行一元和二元数学运算。问题还具有不同的最大可并行任务数，从 2 到 5 不等；并行函数调用之间的汇合次数也不同，以增加问题复杂度。例如，图 3 的 (b) 和 (c) 分别有 2 次和 3 次汇合。基准的 113 个不同示例先由 GPT-4 按上述标准生成，再由人工标注。

J. 24 点游戏与 Tree-of-Thoughts 方法的细节

24 点游戏是一种数学推理游戏：玩家需要对给定的四个数使用加、减、乘、除等基本算术运算，得到 24。规则要求每个给定数字只使用一次。例如，给定 2、4、4、7，一个可能的解为 $4 \times (7 - 4) \times 2 = 24$ 。这一推理基准对 LLM 并不简单：即使 GPT-4 这样的先进模型使用思维链提示，成功率也只有 4% (Yao et al., 2023b)。

ToT 通过多个步骤求解。每一步，称为思路提议器的 LLM 生成若干思路；每个思路是一项部分分解，由两个数及它们之间的算术运算组成。随后，这些思路被送入状态评估器，分别赋予标签 `sure`（确定）、`likely`（可能）或 `impossible`（不可能），表示将运算结果与剩余数字继续组合后得到 24 的可能性。只有有希望得到 24 的思路进入下一步。图 J.6 展示了这一过程。

K. WebShop 实验细节

K.1. WebShop 环境

WebShop 模拟在线购物平台，要求智能体找到最符合给定指令的商品。例如，指令为“我想找一张 queen-size 的黑色床，价格低于 140.00 美元”，智能体就需要准确找到同时满足“queen-size”“黑色”“价格低于 140.00 美元”三个条件的床。每个商品都有对应奖励，根据价格、商品选项及商品页面中的其他细节，衡量它与指令的匹配程度。评测指标包括成功率，即所选商品满足全部要求的回合比例；以及平均得分，即各回合奖励的平均值。

K.2. 基线方法

除 ReAct 外，我们还使用 LASER (Ma et al., 2023) 和 LATS (Zhou et al., 2023a) 作为与 LLMCompiler 比较的基线。LASER (Ma et al., 2023) 采用状态探索方法求解任务。在 WebShop 中，将环境中的不同页面编码为不同状态，例如搜索页、商品页和商品详情子页。动作用于在状态间转移，例如执行搜索查询、选择商品、检查商品详情、进入下一页搜索结果等。因此，WebShop 探索被转换为给定状态空间图上的搜索问题。

⁷译注：原文函数名为 `finish`，但下面定义的说明文字使用 `join`。这里保留两个名称，避免擅自改动提示协议；其描述均指汇合并判断是否回答或重新规划的最终动作。

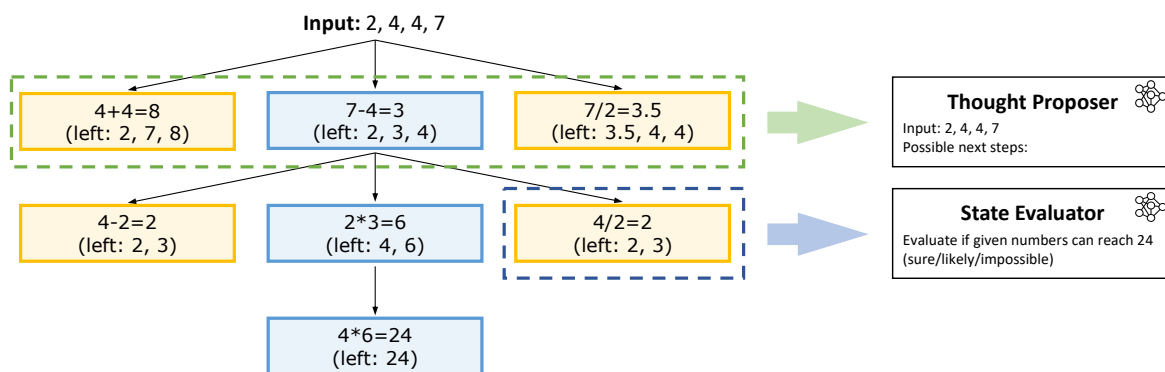


图 J.6. 24 点游戏中的 Tree of Thoughts (ToT) 可视化。每个节点代表不同的提议，从根节点开始，由思路提议器应用单个运算形成分支。状态评估器随后判断各状态达到目标 24 的潜力。ToT 按评价价值保留前五个状态。图内文字译释：Input 为输入；left 为剩余数字；Thought Proposer 为思路提议器；Possible next steps 为可能的下一步；State Evaluator 为状态评估器；评估要求是判断给定数字能否得到 24，标签 sure/likely/impossible 分别表示确定、可能和不可能。

LATS (Zhou et al., 2023a) 使用蒙特卡洛树搜索的一种变体：构建决策树，根据成功的可能性评估潜在动作，并在探索与利用之间权衡来选择动作。随后，智能体根据环境反馈调整策略，从成功与失败中学习，以改进决策过程。这种迭代方法让 LATS 能够处理复杂的在线购物任务，但其穷尽式树搜索使执行速度慢得多。